Troubleshooting Docker Compose Applications

In this train, we'll develop skills to troubleshoot and resolve common issues encountered when managing containerised applications with Docker Compose. We'll examine typical problems and how to address them effectively.

Questions

Question 1

Your application stack is running with Docker Compose, but one of your services is failing to start. What is the best first step in diagnosing the issue?

1. Restart the entire system
2. Run `docker-compose logs service_name`
3. Delete all containers and rebuild everything
4. Run `docker-compose down` and try again

View answer and explanation

> - Correct answer: Run `docker-compose logs service_name`
> - Checking logs is the best first step to understand why a service is failing before taking further action.

Question 2

A container exits immediately after starting. What is the most likely cause?

1. The container has an incorrect network configuration
2. The container process crashed or exited successfully
3. The image is not compatible with Docker Compose
4. The host machine is out of disk space

View answer and explanation

> - Correct answer: The container process crashed or exited successfully

> - If a container exits immediately, it's usually because the command inside completed or failed.

Question 3

After updating your application code, changes are not reflected when restarting the container. What could be the cause?

1. The application service is using a cached image
2. The application needs a different restart policy
3. The Docker daemon is overloaded
4. Docker Compose does not support code updates

View answer and explanation ✖

> - Correct answer: The application service is using a cached image
> - Use `docker-compose up --build` to force a rebuild when updating code.

Question 4

A service cannot connect to another container in the same Docker Compose network. What could be the reason?

1. The service is not defined in the same network
2. Docker Compose does not support inter-container communication
3. The firewall on the host is blocking communication
4. Containers cannot communicate by default

View answer and explanation ✖

> - Correct answer: The service is not defined in the same network
> - Ensure both services are listed under the same network in `docker-compose.yml`.

Question 5

Your database service is failing because the required volume is missing. How can you ensure persistent storage for the database?

1. Use a named volume in `docker-compose.yml`
2. Restart the container and hope the issue resolves
3. Store database files inside the container filesystem
4. Run `docker-compose fix-volumes`

View answer and explanation ✖

> - Correct answer: Use a named volume in `docker-compose.yml`
> - Named volumes ensure persistent storage across container restarts.

Question 6

Your application container is using too much CPU and memory. What is a likely cause of this issue?

1. The container is running in privileged mode
2. The container's image is not optimized
3. The application is handling more requests than the container can handle
4. The container is not being limited by resource constraints

View answer and explanation ✖

> - Correct answer: The container is not being limited by resource constraints
> - Setting proper CPU and memory limits in the Docker Compose file can prevent containers from using excessive resources.

Question 7

After making changes to the `docker-compose.yml` file, you notice the changes are not applied when you run `docker-compose up`. What should you do?

1. Run `docker-compose up --build` to rebuild the containers
2. Run `docker-compose restart` to restart the services
3. Remove the containers and recreate them with `docker-compose down`
4. Make sure the containers are stopped, then start them again with `docker-compose start`

View answer and explanation ✖

> - Correct answer: Run `docker-compose up --build` to rebuild the containers

- This ensures the changes in the Docker Compose file are applied correctly by rebuilding the images.

Question 8

Your application is experiencing slow performance after scaling the service. What should you check first?

1. Check if the application code is optimized for performance
2. Ensure that the Docker Compose file specifies the correct number of replicas
3. Verify that the containers have sufficient resource limits set
4. Check the logs of the affected services for errors

View answer and explanation ✖

- Correct answer: Verify that the containers have sufficient resource limits set
- If containers are running out of resources (CPU, memory), scaling may not solve performance issues.

Question 9

One of your services requires environment variables that are not being picked up in Docker Compose. What is the best way to resolve this?

1. Set the environment variables inside the Dockerfile
2. Ensure that the variables are defined in the `docker-compose.yml` under the `environment` section
3. Pass the environment variables at runtime with `docker-compose run`
4. Use a volume to mount the environment variables file

View answer and explanation ✖

- Correct answer: Ensure that the variables are defined in the `docker-compose.yml` under the `environment` section
- This will ensure that the container has access to the necessary environment variables.

Question 10

You are getting an error stating "cannot connect to the Docker daemon." What is a likely cause of this error?

1. The Docker daemon is not running on the host machine
2. The container image is corrupted
3. Docker Compose is misconfigured
4. The host machine is using an unsupported operating system

View answer and explanation ✖

> - Correct answer: The Docker daemon is not running on the host machine
> - Make sure the Docker daemon is started and running properly on your system.